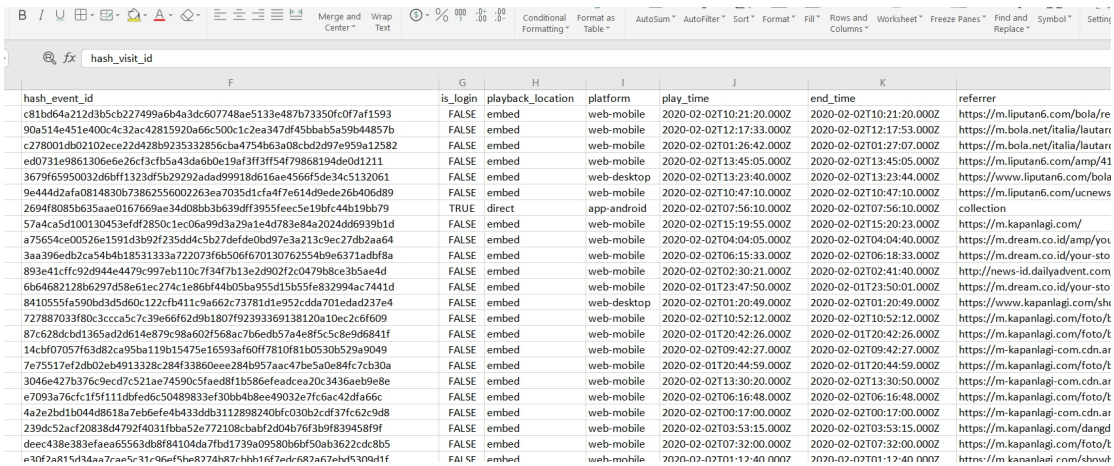
Exploratory Data Analysis (EDA)

To do this data analyst case, I use the top or the smallest dataset, which is the sampling 1% plays dataset. The tools used for this whole questions is Spreadsheet / Excel, Anaconda Navigator, Jupyter Notebook, and Tableau. Mostly, EDA is consisted of things like description of data, handling missing data/values, handling outliers, and understanding relationships and new insights through plots.

In order to start the EDA, I will use Spreadsheet, Jupyter Notebook, and Tableau at the same time. Spreadsheet for easy checking the dataset, and Jupyter Notebook for performing most of the analysis. For now I won't use Tableau. The graph or plot isn't needed yet. First, let's import the basic libraries often used in EDA in Notebook. We can still import libraries as the process is running later.

```
import pandas as pd
import numpy as np
import math
import plotly.express as px
import plotly.graph_objects as go
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

These libraries are the most commonly used for EDA. Pandas is mostly used for dataframes related operations, while numpy and math are for mathemathical operations like square root and matrices. It's not always used, but importing the libraries may come in handy later. The other libraries are mostly for plotting purposes. After that, I load the dataset to the Notebook, while also checking it from Spreadsheet.



hash_visit_id

F	G	H	I	J	K	
hash_event_id	is_login	playback_location	platform	play_time	end_time	referrer
c81bd64a212d3b5cb227499a6b4a3dc607748ae5133e487b73350fc0f7af1593	FALSE	embed	web-mobile	2020-02-02T10:21:20.000Z	2020-02-02T10:21:20.000Z	https://m.liputan6.com/bola/re
90a514e451e400c4c32ac42815920a66c500c1c2ea347df45bbab5a59b44857b	FALSE	embed	web-mobile	2020-02-02T12:17:33.000Z	2020-02-02T12:17:53.000Z	https://m.bola.net/italia/lautar
c278001db02102ece22d428b9235332856cba4754b63a08cbd2d97e959a12582	FALSE	embed	web-mobile	2020-02-02T01:26:42.000Z	2020-02-02T01:27:07.000Z	https://m.bola.net/italia/lautar
ed0731e9861306e6e26cf3cfb5a43da6b0e19af3ff3ff54f79868194de0d1211	FALSE	embed	web-mobile	2020-02-02T13:45:05.000Z	2020-02-02T13:45:05.000Z	https://m.liputan6.com/amp/41
3679f65950032d6bff1323df5b29292adad99918d616ae4566f5de34c5132061	FALSE	embed	web-desktop	2020-02-02T13:23:40.000Z	2020-02-02T13:23:44.000Z	https://www.liputan6.com/bola
9e444d2afa0814830b738625560022 63ea7035d1cfa4f7e614d9ede26b406d89	FALSE	embed	web-mobile	2020-02-02T10:47:10.000Z	2020-02-02T10:47:10.000Z	https://m.liputan6.com/ucnews
2694f8085b635aae0167669ae34d08bb3b639dff3955feec5e19bfc44b19bb79	TRUE	direct	app-android	2020-02-02T07:56:10.000Z	2020-02-02T07:56:10.000Z	collection
57a4ca5d100130453efdf2850c1ec06a99d3a29a1e4d783e84a2024dd6939b1d	FALSE	embed	web-mobile	2020-02-02T15:19:55.000Z	2020-02-02T15:20:23.000Z	https://m.kapanlagi.com/
a75654ce00526e1591d3b92f235dd4c5b27defde0bd97e3a213c9ec27db2aa64	FALSE	embed	web-mobile	2020-02-02T04:04:05.000Z	2020-02-02T04:04:40.000Z	https://m.dream.co.id/amp/you
3aa396edb2ca54b4b18531333a722073f6b506f670130762554b9e6371adbf8a	FALSE	embed	web-mobile	2020-02-02T06:15:33.000Z	2020-02-02T06:18:33.000Z	https://m.dream.co.id/your-sto
893e41cffc92d944e4479c997eb110c7f34f7b13e2d902f2c0479b8ce3b5ae4d	FALSE	embed	web-mobile	2020-02-02T02:30:21.000Z	2020-02-02T02:41:40.000Z	http://news-id.dailyadvent.com
6b64682128b6297d58e61ec274c1e86bf44b05ba955d15b55fe832994ac7441d	FALSE	embed	web-mobile	2020-02-01T23:47:50.000Z	2020-02-01T23:50:01.000Z	https://m.dream.co.id/your-sto
8410555fa590bd3d5d60c122cfb411c9a662c73781d1e952cdda701edad237e4	FALSE	embed	web-desktop	2020-02-02T01:20:49.000Z	2020-02-02T01:20:49.000Z	https://www.kapanlagi.com/sho
727887033f80c3ccca5c7c39e66f62d9b1807f92393369138120a10ec2c6f609	FALSE	embed	web-mobile	2020-02-02T10:52:12.000Z	2020-02-02T10:52:12.000Z	https://m.kapanlagi.com/foto/t
87c628dcbd1365ad2d614e879c98a602f568ac7b6edb57a4e8f5c5c8e9d6841f	FALSE	embed	web-mobile	2020-02-01T20:42:26.000Z	2020-02-01T20:42:26.000Z	https://m.kapanlagi.com/foto/t
14cbf07057f63d82ca95ba119b15475e16593af60ff7810f81b0530b529a9049	FALSE	embed	web-mobile	2020-02-02T09:42:27.000Z	2020-02-02T09:42:27.000Z	https://m-kapanlagi-com.cdn.ar
7e75517ef2db02eb4913328c284f33860eee284b957aac47be5a0e84fc7cb30a	FALSE	embed	web-mobile	2020-02-01T20:44:59.000Z	2020-02-01T20:44:59.000Z	https://m.kapanlagi.com/foto/t
3046e427b376c9ecd7c521ae74590c5faed8f1b586efeadcea20c3436aeb9e8e	FALSE	embed	web-mobile	2020-02-02T13:30:20.000Z	2020-02-02T13:30:50.000Z	https://m-kapanlagi-com.cdn.ar
e7093a76cfc1f5f111dbfed6c50489833ef30bb4b8ee49032e7fc6ac42dfa66c	FALSE	embed	web-mobile	2020-02-02T06:16:48.000Z	2020-02-02T06:16:48.000Z	https://m.kapanlagi.com/foto/t
4a2e2bd1b044d8618a7eb6efe4b433ddb3112898240bfc030b2cdf37fc62c9d8	FALSE	embed	web-mobile	2020-02-02T00:17:00.000Z	2020-02-02T00:17:00.000Z	https://m-kapanlagi-com.cdn.ar
239dc52acf20838d4792f4031fbba52e772108cbabf2d04b76f3b9f839458f9f	FALSE	embed	web-mobile	2020-02-02T03:53:15.000Z	2020-02-02T03:53:15.000Z	https://m.kapanlagi.com/dangd
deec438e383efaea65563db8f84104da7fbd1739a09580b6bf50ab3622cdc8b5	FALSE	embed	web-mobile	2020-02-02T07:32:00.000Z	2020-02-02T07:32:00.000Z	https://m.kapanlagi.com/foto/t
e30f2a815d34aa7cae5c31c96ef5be8274b87cbbb16f7edc682a67ebd5309d1f	FALSE	embed	web-mobile	2020-02-02T01:12:40.000Z	2020-02-02T01:12:40.000Z	https://m.kapanlagi.com/show

```
df = pd.read_csv('part-00000-0dd5f001-57d7-4ec1-a3f0-53ea28bba9c3-c000.csv',error_bad_lines=False)
df.head()
```

```
b'Skipping line 23509: expected 41 fields, saw 43\n'
b'Skipping line 36946: expected 41 fields, saw 42\nSkipping line 43788: expected 41 fields, saw 42\nSkipping line 45437: expect
ed 41 fields, saw 49\n'
b'Skipping line 52141: expected 41 fields, saw 42\n'
b'Skipping line 73470: expected 41 fields, saw 42\nSkipping line 75103: expected 41 fields, saw 42\n'
b'Skipping line 87928: expected 41 fields, saw 42\n'
b'Skipping line 98453: expected 41 fields, saw 42\nSkipping line 98771: expected 41 fields, saw 42\nSkipping line 101047: expec
ted 41 fields, saw 42\nSkipping line 104676: expected 41 fields, saw 42\n'
C:\ProgramData\Anaconda3\lib\site-packages\IPython\core\interactiveshell.py:3071: DtypeWarning: Columns (20) have mixed types.S
pecify dtype option on import or set low_memory=False.
  has_raised = await self.run_ast_nodes(code_ast.body, cell_name,
```

	hash_content_id	hash_play_id	hash_visit_id
0	0096dafb1049ee942e8e7cbad5abf4a46dc92e3995caac...	0162f124f5bd61592d9bca6aaa3b1b6097a00b62ef33c9...	27dfffe2bb74f8767caeb64ac4c92e9eaf4b11a28ef7e1...
1	01ddda2f8667719625bd0afa58537fd95a166c93233b8b...	5f6c722782c919a9cb6254e94086462a6c34794dc741b6...	90b2b454e70a2c13cf9e8ca987d7c4f914c7d819532c5d...

The dataset can open normally in Spreadsheet, but apparently there are errors when I tried to load it in Notebook. I use a simple command to skip the lines with errors, assuming with how many rows the datas are, the skipped lines is just a little fractions of it. Next I will use the shape() command to basically know how many rows and columns in this dataset.

```
df.shape
```

```
(106811, 41)
```

Command above means this dataframe have 106811 rows and 41 columns. This, of course, also counting all of the missing value. The command below shows each of column names, sorted from the left.

```
df.columns
```

```
Index(['hash_content_id', 'hash_play_id', 'hash_visit_id', 'hash_watcher_id',
       'hash_film_id', 'hash_event_id', 'is_login', 'playback_location',
       'platform', 'play_time', 'end_time', 'referrer', 'average_bitrate',
       'bitrate_range', 'total_bytes', 'buffer_duration', 'referrer_group',
       'completed', 'utm_source', 'utm_medium', 'utm_campaign', 'player_name',
       'has_ad', 'flash_version', 'os_name', 'os_version', 'browser_name',
       'browser_version', 'app_name', 'autoplay', 'is_premium', 'app_version',
       'city', 'play_duration', 'content_type', 'stream_type', 'title',
       'category_name', 'film_title', 'season_name', 'genre_name'],
      dtype='object')
```

Next is, I check for each column using info() command. This command will show several information related to the dataframe. For example, the name of each column, their non-null count, and the data type for each column.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 106811 entries, 0 to 106810
Data columns (total 41 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   hash_content_id    106811 non-null  object
 1   hash_play_id       106811 non-null  object
 2   hash_visit_id      106811 non-null  object
 3   hash_watcher_id    106811 non-null  object
 4   hash_film_id       5818 non-null    object
 5   hash_event_id      106811 non-null  object
 6   is_login           106811 non-null  bool
 7   playback_location  106811 non-null  object
 8   platform           106811 non-null  object
 9   play_time          106811 non-null  object
 10  end_time           106811 non-null  object
 11  referrer           101050 non-null  object
 12  average_bitrate    101865 non-null  float64
 13  bitrate_range      106811 non-null  object
 14  total_bytes        106811 non-null  int64
 15  buffer_duration    106811 non-null  float64
 16  referrer_group     106811 non-null  object
 17  completed          67874 non-null   object
 18  utm_source         87 non-null      object
 19  utm_medium         87 non-null      object
 20  utm_campaign       38 non-null      object
 21  player_name        103544 non-null  object
 22  has_ad             106811 non-null  bool
 23  flash_version      71961 non-null   object
 24  os_name            73086 non-null   object
 25  os_version         106406 non-null  object
 26  browser_name       73145 non-null   object
 27  browser_version    70773 non-null   object
 28  app_name           106811 non-null  object
 29  autoplay           67851 non-null   object
 30  is_premium         106811 non-null  bool
 31  app_version        34682 non-null   object
 32  city               0 non-null       float64
 33  play_duration      106811 non-null  int64
 34  content_type       106811 non-null  object
 35  stream_type        38937 non-null   object
 36  title              106811 non-null  object
 37  category_name      67874 non-null   object
 38  film_title         5816 non-null    object
 39  season_name        5818 non-null    object
 40  genre_name         5225 non-null    object
dtypes: bool(3), float64(3), int64(2), object(33)
memory usage: 31.3+ MB
```

We can see from above, the dataset have 3 boolean columns, 3 float columns, 2 int columns, and 33 object columns. Object data type also can be called string data type. Remember before the dataframe have 106811 rows? If the non null count is not the same as that value, then there are missing values in that column. We will elaborate it further later, for now we check the dataframe with the describe() command.

```
df.describe()
```

	average_bitrate	total_bytes	buffer_duration	city	play_duration
count	101865.000000	1.068110e+05	1.068110e+05	0.0	106811.000000
mean	199406.240230	7.765342e+07	2.272422e+05	NaN	425.431847
std	141397.648432	4.683612e+08	7.416347e+07	NaN	1541.638614
min	144.000000	0.000000e+00	0.000000e+00	NaN	0.000000
25%	633.000000	7.500000e+02	0.000000e+00	NaN	4.000000
50%	300000.000000	5.062500e+05	1.000000e+00	NaN	41.000000
75%	300000.000000	1.125000e+07	6.000000e+00	NaN	120.000000
max	300000.000000	3.092062e+10	2.423807e+10	NaN	82455.000000

Describe is basically describing the correspondent column in statistical manners. This includes the count, mean, standard deviation (std), minimum value, the quartiles, and maximum value. By default, because describe is statistical, it only includes the column with number data type. We can change the command to include all other column, but I don't think it's necessary in this case.

We can see from info and describe there are several columns with missing value. There are many reasons for missing data. For example, user did not want to fill data due to privacy issues. There are other reasons too. Loss of data while transferring or insufficient information to fill a particular column is also a common reason. Now we will check for column with null values.

```
df.isnull().values.any()
```

```
True
```

This command just check the obvious, what we already know several seconds ago. That there is null or missing value in the dataset. This command, however, at least helps when the analyst is not careful enough reading the numbers before. The command below will count each of the column missing values.

```
df.isnull().sum()
```

From this command, we can see how many columns and the nummber of missing values. We can see several columns like hash_film_id, referrer, average_bitrate, completed, utm_source, utm_medium, utm_campaign, player_name, flash_version, os_name, os_version, browser_name, browser_version, autoplay, app_version, city, stream_type, category_name, film_title, season_name and genre_name all have missing value. There are several columns which have a lot of missing values. Those columns also mostly have object data type, so I think it's better to just drop them.

```
df = df.drop(columns = 'city', axis = 1)
```

```
df = df.drop(['hash_film_id', 'utm_source', 'utm_medium', 'utm_campaign','film_title', 'season_name', 'genre_name'], axis = 1)
```

Command above is for dropping columns. The first one is for dropping a single column, while the second is for multiple columns. All the columns that dropped have around 100000 missing values. For the other columns with missing values, if they can't be replaced then it's simply faster to drop them. However, I think the overall objective of this study case is to explore and analyze, thus, some columns probably will have information for later.

```
hash_content_id          0
hash_play_id             0
hash_visit_id            0
hash_watcher_id          0
hash_film_id        100993
hash_event_id            0
is_login                 0
playback_location        0
platform                 0
play_time                0
end_time                 0
referrer              5761
average_bitrate       4946
bitrate_range            0
total_bytes              0
buffer_duration          0
referrer_group           0
completed            38937
utm_source          106724
utm_medium          106724
utm_campaign        106773
player_name           3267
has_ad                   0
flash_version        34850
os_name              33725
os_version             405
browser_name         33666
browser_version      36038
app_name                 0
autoplay             38960
is_premium               0
app_version          72129
city                106811
play_duration            0
content_type             0
stream_type          67874
title                    0
category_name        38937
film_title          100995
season_name         100993
genre_name          101586
dtype: int64
```

After this I will check several columns which I suspect have duplicate value in them.

```
df.duplicated(subset=['referrer']).any()
```

```
True
```

```
df.duplicated(subset=['hash_play_id']).any()
```

```
False
```

```
df.duplicated(subset=['hash_content_id']).any()
```

```
True
```

```
df.duplicated(subset=[ 'hash_visit_id']).any()
```

```
True
```

```
df.duplicated(subset= ['hash_watcher_id']).any()
```

```
True
```

```
df.duplicated(subset=['hash_event_id']).any()
```

```
False
```

```
df.duplicated(subset=['hash_film_id']).any()
```

```
True
```

```
df.duplicated(subset=['bitrate_range']).any()
```

```
True
```

```
df.duplicated(subset=['title']).any()
```

```
True
```

Unsurprisingly, there are some columns with duplicated values. There's also columns with unique values. This actually will contribute in later insight (or question 2 if you will). The last thing to do before wrapping question 1 and move to graph analyzing for question 2, is checking the correlation for the dataset. The rule of thumb is correlation can only be calculated with number data type. The command will automatically choose the column with number data type and calculate the correlations.

```
df.corr()
```

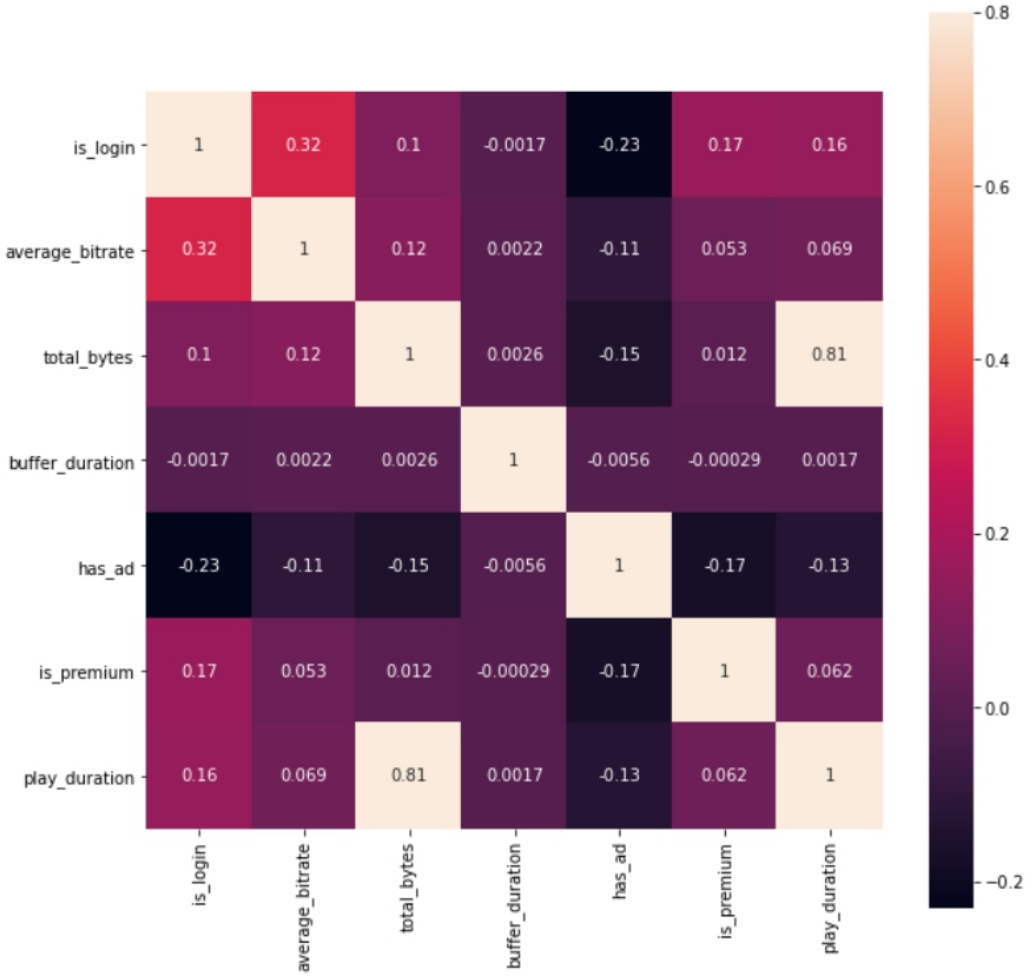
	is_login	average_bitrate	total_bytes	buffer_duration	has_ad	is_premium	play_duration
is_login	1.000000	0.319692	0.101245	-0.001672	-0.231610	0.167211	0.158977
average_bitrate	0.319692	1.000000	0.120405	0.002231	-0.105269	0.053275	0.069359
total_bytes	0.101245	0.120405	1.000000	0.002588	-0.154921	0.012104	0.812459
buffer_duration	-0.001672	0.002231	0.002588	1.000000	-0.005586	-0.000286	0.001663
has_ad	-0.231610	-0.105269	-0.154921	-0.005586	1.000000	-0.168976	-0.129162
is_premium	0.167211	0.053275	0.012104	-0.000286	-0.168976	1.000000	0.062105
play_duration	0.158977	0.069359	0.812459	0.001663	-0.129162	0.062105	1.000000

I have known some method for easier reading the correlation value. The function below is just for changing the text color. If it's positive then the text's color is black. If negative, it's red. However, don't assume that the negative value means auto dropping the column (if it's used for modelling). Correlation actually have positive and negative influences. If the correlation value is positive, then they affect each other positively, for example when a column's value is up then the other is also rising. And vice versa. For the negative values it's the opposite. If one is rising, the the other will fall (heheh). It's a good mindset to check which column has positive or negative impact towards the target column.

```
def color_negative_red(val):
    color = 'red' if val < 0 else 'black'
    return 'color: %s' % color
```

```
df.corr().style.applymap(color_negative_red)
```

	is_login	average_bitrate	total_bytes	buffer_duration	has_ad	is_premium	play_duration
is_login	1.000000	0.319692	0.101245	-0.001672	-0.231610	0.167211	0.158977
average_bitrate	0.319692	1.000000	0.120405	0.002231	-0.105269	0.053275	0.069359
total_bytes	0.101245	0.120405	1.000000	0.002588	-0.154921	0.012104	0.812459
buffer_duration	-0.001672	0.002231	0.002588	1.000000	-0.005586	-0.000286	0.001663
has_ad	-0.231610	-0.105269	-0.154921	-0.005586	1.000000	-0.168976	-0.129162
is_premium	0.167211	0.053275	0.012104	-0.000286	-0.168976	1.000000	0.062105
play_duration	0.158977	0.069359	0.812459	0.001663	-0.129162	0.062105	1.000000

Lastly for this question, is how to show the heatmap. The heatmapn is basically used for visualizing the correlation again, but with more flair. Heatmap is one of the prominent graph for Seaborn libraries. The other graph and in depth analysis for remaining columns will go to question 2, along with the usage of Tableau.

```
C_mat = df.corr()
fig = plt.figure(figsize = (10,10))

sns.heatmap(C_mat, vmax = .8, square = True,annot = True)
plt.show()
```